

# NutriLen - Documentación Técnica y Funcional

---

## NutriLen - Documentacion Tecnica y Funcional (Version Ampliada)

### 1. Introduccion

Este documento consolida la documentacion tecnica y funcional de NutriLen con trazabilidad sobre:

- Codigo fuente (`frontend`, `backend`, `database`).
- Documentacion Markdown previa en `docs`.
- PDF del proyecto, incluyendo matriz de RNF ISO 25010 (21 requerimientos).

### 2. Objetivos Generales

- Describir el sistema end-to-end con enfoque de arquitectura, funcionalidad y operacion.
- Verificar el cumplimiento real de requerimientos funcionales y no funcionales.
- Identificar brechas tecnicas y fases pendientes para cierre de proyecto.

### 3. Objetivos Especificos

- Documentar modulos, endpoints, DTOs y reglas de negocio.
- Modelar datos, integridad y relaciones.
- Evaluar seguridad, calidad, testing, accesibilidad y despliegue.
- Proponer plan de cierre orientado a evidencia.

### 4. Contexto

- Dominio: evaluacion sensorial de medallones de lenteja.
- Actores: participante (encuesta publica), administrador (dashboard y reportes), equipo tecnico.

- Stack: Next.js + NestJS + PostgreSQL (Supabase) + Clerk + exportador Excel Python.

## 5. Desarrollo

### 5.1 Resumen

- Nombre: NutriLen.

- Problema que resuelve: captura estructurada de encuestas, analisis estadistico y exportabilidad para toma de decisiones.

- Alcance implementado:

1. Registro de encuesta (POST /api/v1/encuestas).
2. Acceso admin protegido (GET /api/v1/administrador/me).
3. Consulta estadistica filtrada (GET /api/v1/estadisticas).
4. Exportacion Excel (GET /api/v1/estadisticas/excel).

- Beneficio central: trazabilidad de resultados sensoriales con capa administrativa protegida.

### 5.2 Descripcion General del Sistema

Flujo principal:

1. Participante completa encuesta multi-step en frontend.
2. Frontend envia payload al backend.
3. Backend valida DTO y persiste en PostgreSQL.
4. Administrador autenticado consulta estadisticas.
5. Administrador exporta resultados en Excel/PDF o simplemente observa las estadisticas.

Modulos backend:

- `encuesta`: alta y validacion de encuestas.
- `administrador`: autenticacion/autorizacion administrativa.
- `estadistica`: consulta con filtros y exportador.

- `database`: pool PostgreSQL y ejecucion de queries.

### 5.3 Arquitectura del Sistema

- Arquitectura por capas:

- Presentacion: Next.js.
- Aplicacion/API: NestJS.
- Datos: PostgreSQL.
- Servicio auxiliar: Python exporter.

- Patrones de disenno aplicados:

- Repository:

- Concepto: organiza todo lo relacionado con base de datos en un solo lugar.
- Aplicacion en NutriLen: en `encuesta` y `estadistica` se separo la consulta a datos de la logica general.

- Motivo de uso: queriamos que los cambios en base de datos no obliguen a tocar todo el sistema.

- Strategy:

- Concepto: permite tener varias formas de filtrar y elegir la que corresponde segun el caso.

- Aplicacion en NutriLen: filtros de estadisticas por `diet`, `sex` y rango de fechas.

- Motivo de uso: asi podemos agregar filtros nuevos sin romper lo que ya funciona.

- Decorator + Guard (NestJS):

- Concepto: agrega una verificacion antes de entrar a ciertas rutas.

- Aplicacion en NutriLen: `@UseGuards(AdminGuard)` en rutas de administrador.

- Motivo de uso: dejar bien claro que no cualquiera puede ver estadisticas internas.

- Factory:

- Concepto: centraliza la eleccion de que herramienta usar segun el tipo de salida.

- Aplicacion en NutriLen: para exportar, el sistema decide si usa generador de PDF o de Excel.

- Motivo de uso: en el futuro se pueden sumar mas formatos sin rehacer todo.
- Facade:
  - Concepto: ofrece una puerta unica para acceder a funciones que estaban dispersas.
  - Aplicacion en NutriLen: ``frontend/src/lib/api.ts`` concentra llamadas HTTP y acceso a utilidades.
  - Motivo de uso: ordenar el codigo y facilitar el trabajo cuando se incorporan cambios.
- Observer (reactividad con hooks):
  - Concepto: cuando cambian los datos, la pantalla se actualiza sola.
  - Aplicacion en NutriLen: hooks de filtros/estadisticas y actualizacion del dashboard en tiempo real.
  - Motivo de uso: mejorar la experiencia y evitar actualizaciones manuales en cada componente.

Ventajas y desventajas de los patrones elegidos:

- Ventajas:
  - ``Repository``: ordena el acceso a datos y evita mezclar reglas de negocio con consultas.
  - ``Strategy``: permite sumar filtros nuevos sin reescribir toda la logica de estadisticas.
  - ``Factory``: simplifica la exportacion en PDF/Excel y facilita agregar formatos futuros.
  - ``Facade``: deja un punto de entrada mas claro para consumo de API desde frontend.
  - ``Observer``: mejora la reactividad del dashboard cuando cambian filtros o datos.
  - ``Decorator/Guard``: refuerza seguridad al proteger rutas admin de forma consistente.
- Desventajas:
  - Aumenta la cantidad de archivos y puede parecer mas complejo al inicio.
  - Exige disciplina de equipo para respetar contratos e interfaces.
  - Si no hay tests automatizados, el beneficio de estos patrones se aprovecha menos.

Justificacion respecto de otros patrones de la asignatura:

- Se priorizaron los patrones que resolvían necesidades reales del alcance actual (filtros, seguridad, exportes y organización de código).
- Patrones como `State`, `Visitor`, `Mediator` o `Chain of Responsibility` no eran prioritarios en esta iteración porque agregaban complejidad sin un beneficio claro para los problemas concretos del proyecto.

#### 5.4 Tecnologías Utilizadas

- Frontend: Next.js `16.2.6`, React `19.2.4`, Tailwind v4, Recharts, jsPDF.
- Backend: NestJS `10.4.6`, `pg 8.16.3`, class-validator, class-transformer.
- Auth: Clerk (`@clerk/nextjs`, `@clerk/backend`).
- DB: PostgreSQL (Supabase).
- Exportes: Python + XlsxWriter `3.2.0`.
- CI: GitHub Actions (lint/build frontend y backend).

#### 5.5 Estructura del Repositorio

- `/frontend`: UI, hooks, cliente API y exportadores cliente.
- `/backend`: API NestJS, endpoint serverless Vercel, exportador Python.
- `/database`: `schema.sql`.
- `/docs`: documentación técnica/académica y registro de iteraciones.
- `/tests`: sin suites activas (solo `.gitkeep`).
- `/.github/workflows/\*\*`: workflows de CI configurados.

#### 5.6 Modelo de Datos

El modelo de datos separa dos responsabilidades principales:

- usuarios: identidad y autorización de acceso administrativo.
- encuestas: respuestas funcionales de la evaluación sensorial.
- Esta separación permite mantener la encuesta pública/anónima, y a la vez proteger el acceso al panel interno del administrador.

#### `public.usuarios`

- PK: `id` UUID.
- Campos clave: `email` unique, `rol` check (`admin|cliente`), `activo`.
- Uso: autorizacion admin DB-first.

#### `public.encuestas`

- PK: `id` UUID.
- FK: `usuario\_id` -> `usuarios.id` (`ON DELETE SET NULL`).
- Campos de negocio: sexo, dieta, atributos sensoriales 1..5, aceptacion 1..5, liked, consume\_again, recommend 1..5, comentarios.
- Indices: fecha, sexo, dieta, usuario\_id.

Integridad:

- Validaciones duales: DTO backend + constraints SQL para escalas numericas.
- Coherencia funcional: las escalas sensoriales se validan antes de guardar y se vuelven a validar en base de datos.
- Trazabilidad: los indices por fecha/sexo/dieta permiten consultas estadisticas rapidas y verificables.

### 5.7 Reglas de Negocio Implementadas

1. Solo admin accede a rutas administrativas/estadisticas.
2. Admin valido = token Clerk + usuario con `rol='admin'` y `activo=true`.
3. Encuesta valida enums y rangos obligatorios.
4. Fecha por defecto en backend cuando no se informa.
  - Que significa: si el frontend no envia `date`, el backend asigna fecha/hora actual del servidor.
  - Para que se hace: evita registros sin fecha y mantiene consistencia.
  - Criterio esperado: toda encuesta persistida debe tener `fecha` no nula.

5. Filtro `to` normalizado al fin del día.

- Que significa: si el usuario filtra hasta `YYYY-MM-DD`, el backend interpreta hasta `YYYY-MM-DD 23:59:59`.

- Para que se hace: incluir todas las encuestas de ese día y evitar cortes involuntarios.

- Criterio esperado: el límite superior del rango siempre incluye el día completo.

6. Exporte Excel usa datos filtrados del backend.

### 5.8 Requerimientos Funcionales (RF)

Estados: `Cumple`, `Parcial`, `No cumple` (contra implementación actual).

1. RF-01 Visualización de información del producto: **Cumple**.

Evidencia: existe Home con contenido visual; se verifica en código evidencia completa y estructurada de todos los bloques exigidos (ingredientes + info nutricional + objetivos en formato formal).

2. RF-02 Navegación entre Inicio/Encuesta/Estadísticas: **Cumple**.

Evidencia: navbar y rutas activas en frontend (`/`, `/encuesta`, `/administrador`).

3. RF-03 Registro de sexo biológico y tipo de dieta: **Cumple**.

Evidencia: paso 1 de encuesta + validación DTO backend.

4. RF-04 Generación automática de fecha: **Cumple**.

Evidencia: frontend crea `date` y backend aplica default si no llega.

5. RF-05 Evaluación sensorial descriptiva (6 atributos 1..5): **Cumple**.

Evidencia: sliders + validaciones `@Min(1) @Max(5)`.

6. RF-06 Registro de observaciones descriptivas: **Cumple**.

Evidencia: `descriptiveComments` en UI, DTO, DB y estadísticas.

7. RF-07 Evaluación afectiva (escala): **Cumple**.

Evidencia: `acceptance` escala 1..5 implementada.

8. RF-08 Registro de satisfacción general (Si/No): **Cumple**.

Evidencia: campo `liked` con enum `si|no`.

9. RF-09 Registro de observaciones afectivas: **\*\*Cumple\*\***.

Evidencia: `affectiveComments` implementado end-to-end.

10. RF-10 Encuesta multi-step (3 pasos): **\*\*Cumple\*\***.

Evidencia: flujo de tres etapas en `/encuesta`.

11. RF-11 Persistencia entre pasos: **\*\*Cumple\*\***.

Evidencia: estado React conserva datos mientras navega entre pasos.

12. RF-12 Navegacion entre pasos (Continuar/Volver): **\*\*Cumple\*\***.

Evidencia: botones y handlers `next/prev`.

13. RF-13 Confirmacion de envio: **\*\*Cumple\*\***.

Evidencia: toast de exito + vista final de agradecimiento.

14. RF-14 Almacenamiento en base de datos: **\*\*Cumple\*\***.

Evidencia: `INSERT INTO public.encuestas` en repositorio backend.

15. RF-15 Generacion de estadisticas: **\*\*Cumple\*\***.

Evidencia: consulta backend + procesamiento frontend (ViewModel dashboard).

16. RF-16 Dashboard estadistico (KPIs y distribuciones): **\*\*Parcial\*\***.

Evidencia: total, score, satisfaccion y distribuciones implementadas; "encuestas en curso" no surge como metrica persistida real en backend.

17. RF-17 Grafico radar/telarana: **\*\*Cumple\*\***.

Evidencia: seccion sensorial y radar en dashboard/reportes.

18. RF-18 Interaccion dinamica del radar (seleccionar/deseleccionar atributos): **\*\*Cumple\*\***.

Evidencia: no se detecta control explicito de toggle de atributos del radar en la implementacion actual.

19. RF-19 Tooltips estadisticos: **\*\*Parcial\*\***.

Evidencia: libreria de graficos soporta tooltips, pero no hay evidencia clara y uniforme de tooltips personalizados en todas las metricas requeridas.

20. RF-20 Exportacion de reportes PDF y Excel: **\*\*Cumple\*\***.

Evidencia: `exportarPDF` y `exportarExcel` implementados.



21. RF-21 Actualizacion de estadisticas por boton refresco: **Cumple**.

Evidencia: accion `refresh` en panel admin.

22. RF-22 Cambio de tema visual (claro/oscuro): **Cumple**.

Evidencia: no se observa conmutador de tema implementado en frontend actual.

23. RF-23 Diseno responsive: **Parcial**.

Evidencia: clases responsive presentes; falta evidencia formal de pruebas multiplataforma.

24. RF-24 Validacion de formularios obligatorios: **Cumple**.

Evidencia: validaciones en frontend por paso + DTO backend con `class-validator`.

25. RF-25 Animaciones e interaccion visual: **Cumple**.

Evidencia: loaders/transiciones/feedback visual en encuesta y dashboard.

Resumen RF:

- Cumple: 22

- Parcial: 3

- No cumple: 0

### 5.9 Requerimientos No Funcionales (RNF) - Matriz de 21 RNF

Fuente: `Grupo5-RNF-ISO25010 - Hoja 1 (1).pdf` (texto extraido y contrastado con codigo).

Estados: `Cumple`, `Parcial`, `No cumple` (evidencia tecnica actual).

1. RNF-01 Tiempo de respuesta  $\leq 3s$ : **Parcial**.

Evidencia: app funcional; no hay metricas ni pruebas de performance.

2. RNF-02 Carga eficiente de imagenes ( $<2s$ ): **Parcial**.

Evidencia: uso de assets optimizados; no hay medicion formal.

3. RNF-03 Soportar  $\geq 30$  encuestas concurrentes: **Parcial** (evidencia).

Evidencia: no existen pruebas de carga/concurrencia.

4. RNF-04 Escalabilidad funcional: **\*\*Cumple\*\***.

Evidencia: arquitectura modular y patrones extensibles.

5. RNF-05 Disponibilidad durante jornada: **\*\*Parcial\*\***.

Evidencia: healthcheck/CI; sin SLA/monitoreo operativo.

6. RNF-06 Persistencia de respuestas: **\*\*Cumple\*\***.

Evidencia: insert SQL + confirmacion de envio en frontend tras respuesta OK.

7. RNF-07 Recuperacion ante fallos de envio: **\*\*Cumple\*\***.

Evidencia: manejo de error y posibilidad de reintento sin perder estado local del formulario.

8. RNF-08 Integridad de datos: **\*\*Cumple\*\***.

Evidencia: DTO validations + checks SQL.

9. RNF-09 Autenticacion administrativa (Clerk): **\*\*Cumple\*\***.

Evidencia: guard backend valida Bearer token con Clerk.

10. RNF-10 Autorizacion por rol: **\*\*Cumple\*\***.

Evidencia: consulta DB por `rol='admin' AND activo=true`.

11. RNF-11 Encuesta anonima: **\*\*Cumple\*\***.

Evidencia: ruta publica sin login requerido.

12. RNF-12 Proteccion de datos almacenados: **\*\*Parcial\*\***.

Evidencia: controles de acceso backend; sin evidencia de RLS/auditoria completa.

13. RNF-13 Usabilidad multi-step: **\*\*Cumple\*\***.

Evidencia: encuesta con pasos y botones de navegacion claros.

14. RNF-14 Accesibilidad visual (Lighthouse >=70): **\*\*No cumple\*\*** (evidencia).

Evidencia: no hay reporte Lighthouse versionado.

15. RNF-15 Responsive (mobile/tablet/desktop): **\*\*Parcial\*\***.

Evidencia: clases responsivas; sin suite de pruebas cross-device formal.

16. RNF-16 Consistencia visual/paleta NutriLen: **\*\*Cumple\*\***.

Evidencia: variables y lineamiento visual consistentes entre vistas principales.

17. RNF-17 Compatibilidad Chrome/Edge/Firefox: **\*\*No cumple\*\*** (evidencia).

Evidencia: no hay matriz de pruebas por navegador.

18. RNF-18 Integracion frontend-backend-BD: **\*\*Cumple\*\***.

Evidencia: contrato DTO/API operativo y consultas reales en dashboard.

19. RNF-19 Mantenibilidad modular: **\*\*Cumple\*\***.

Evidencia: separacion en modulos, hooks, repositorios y servicios.

20. RNF-20 Despliegue/operacion Vercel-Supabase: **\*\*Parcial\*\***.

Evidencia: configuracion Vercel presente; falta evidencia completa de operacion en todos los entornos requeridos.

21. RNF-21 Registro de eventos relevantes: **\*\*No cumple\*\*** (evidencia).

Evidencia: no hay capa formal de logging/auditoria de eventos administrativos y fallos.

Resumen RNF:

- Cumple: 11

- Parcial: 7

- No cumple: 3

## 5.10 Casos de Uso

### #### Actor Participante

- Objetivo: responder encuesta sensorial de forma completa y valida.

- Precondicion: acceso a `/encuesta` y disponibilidad de backend.

- Flujo principal:

1. Ingresa datos generales.

2. Completa evaluacion descriptiva (6 atributos + observaciones).

3. Completa evaluacion afectiva y envia.

4. Recibe confirmacion de envio exitoso.

- Flujos alternativos:

- Faltan campos obligatorios: el sistema muestra validaciones y bloquea avance/envio.
- Error de red/envio: el sistema informa falla y permite reintentar.
- Postcondicion:
  - Exito: encuesta guardada y disponible para estadisticas.
  - Falla: usuario informado sin perdida de contexto del formulario.

#### #### Actor Administrador

- Objetivo: consultar resultados y exportar reportes.
- Precondicion: sesion valida en Clerk + usuario `admin` activo en BD.
- Flujo principal:
  1. Ingresa al panel `/administrador`.
  2. El sistema valida autenticacion y rol.
  3. Aplica filtros de fecha/sexo/dieta.
  4. Consulta KPIs y graficos.
  5. Exporta resultados en Excel o PDF.
- Flujos alternativos:
  - Usuario sin rol admin: acceso denegado.
  - Sin datos para el filtro aplicado: se muestran metricas vacias sin error de aplicacion.
- Postcondicion: analitica consultada y/o reporte exportado correctamente.

#### 5.11 API Implementada

- `GET /` estado base backend.
- `GET /api/v1/health` healthcheck.
- `POST /api/v1/encuestas` alta encuesta.
- `GET /api/v1/admin/me` verificacion admin.
- `GET /api/v1/estadisticas` consulta filtrada.

- `GET /api/v1/estadisticas/excel` exporte Excel.

### 5.12 Seguridad

Implementado:

- Verificacion Clerk backend-side.
- Autorizacion por rol en DB.
- SQL parametrizado.

Brechas:

- Rate limiting.
- Politica CORS por entorno.
- Logging de accesos/fallos y trazabilidad de seguridad.

### 5.13 Testing

Estado:

- Sin tests unitarios/integracion/e2e implementados.
- Cobertura automatizada estimada: 0%.

Impacto:

- RNF de rendimiento, compatibilidad y observabilidad quedan sin evidencia objetiva.

### 5.14 Accesibilidad

- Hay base UI utilizable, pero no existe evidencia formal de cumplimiento WCAG/Lighthouse.
- Requiere baseline y plan de correccion verificable.

### 5.15 Despliegue

Variables relevantes:

- `DATABASE\_URL`
- `CLERK\_SECRET\_KEY`
- `NEXT\_PUBLIC\_CLERK\_PUBLISHABLE\_KEY`
- `NEXT\_PUBLIC\_API\_URL`
- `NEXT\_PUBLIC\_DEV\_LOCAL\_FALLBACK`
- `PYTHON\_EXECUTABLE`
- `EXCEL\_PYTHON\_EXPORT\_URL`
- `EXCEL\_EXPORT\_INTERNAL\_TOKEN`

CI:

- Workflows frontend/backend con lint + build.

#### 5.16 Diagrama UML de Clases (PlantUML)

(agregar diagrama al final del documento)

#### 5.17 Plan de Mejoras por Fases (Integrado)

Esta hoja de ruta unifica puntos de mejora y fases pendientes, con foco en cerrar brechas funcionales y de calidad de forma ordenada.

Fase 1 - Calidad tecnica base (QA Basica)

- Implementar tests unitarios backend (DTOs, servicios, guardias).
- Implementar tests de integracion para endpoints criticos (`encuestas`, `estadisticas`, `admin`).
- Resultado esperado: reduccion de errores regresivos en cambios frecuentes.

Fase 2 - Flujo funcional end-to-end

- Implementar pruebas E2E para envio de encuesta, acceso admin y exportes.
- Incluir casos de acceso no autorizado y manejo de errores de red.

- Resultado esperado: validacion de procesos reales de usuario de punta a punta.

### Fase 3 - Seguridad y robustez operativa

- Hardening: rate limiting, CORS estricto por entorno, manejo de errores estandarizado.
- Observabilidad: logging/auditoria de eventos criticos (RNF-21).
- Resultado esperado: mejor trazabilidad y menor riesgo de incidentes.

### Fase 4 - Rendimiento y compatibilidad

- Ejecutar pruebas de carga para RNF-01 y RNF-03.
- Formalizar matriz cross-browser (Chrome/Edge/Firefox) y matriz responsive.
- Resultado esperado: evidencia objetiva de performance y compatibilidad.

### Fase 5 - Accesibilidad y cierre documental

- Ejecutar auditoria Lighthouse/WCAG y plan de correccion.
- Consolidar matriz final RF/RNF con evidencia de cumplimiento.
- Resultado esperado: cierre de calidad listo para entrega academica.

## 6. Anexo A - Requerimientos Normalizados IEEE 29148 (Lista para Defensa)

Objetivo de esta seccion: presentar el catalogo completo de requerimientos en formato tecnico normalizado.

Formato aplicado por requerimiento: `ID`, `Requerimiento normalizado`, `Criterio de aceptacion`, `Prueba asociada`.

### 8.1 Requerimientos Funcionales (25)

| ID | Requerimiento normalizado | Criterio de aceptacion | Prueba asociada |

|---|---|---|---|

| RF-01 | El sistema debera mostrar en Home: imagen, descripcion, ingredientes, informacion nutricional y objetivos del proyecto. | Los 5 bloques son visibles y accesibles desde Home. | Prueba UI manual + checklist de contenido. |

| RF-02 | El sistema debera permitir navegar entre Inicio, Encuesta y Estadisticas mediante barra de navegacion. | Las 3 rutas son alcanzables en <=2 clics. | Prueba E2E de navegacion. |

| RF-03 | El participante debera registrar sexo biologico y tipo de dieta antes de continuar. | No se avanza al paso 2 sin ambos campos validos. | Prueba funcional de validaciones por paso. |

| RF-04 | El sistema debera registrar automaticamente la fecha de encuesta si no es enviada por el cliente. | Toda encuesta persistida contiene fecha valida. | Test backend de default de fecha. |

| RF-05 | El sistema debera capturar 6 atributos descriptivos con escala entera 1..5. | Ningun valor fuera de rango se persiste. | Prueba API con casos validos/invalidos. |

| RF-06 | El sistema debera permitir registrar observaciones descriptivas opcionales. | El campo acepta texto y se persiste sin truncado indebido. | Prueba UI/API de persistencia de comentarios. |

| RF-07 | El sistema debera registrar aceptacion general con escala afectiva 1..5. | Se persiste un entero entre 1 y 5. | Prueba API de validacion de rango. |

| RF-08 | El sistema debera registrar respuesta Si/No a "Te gusto el producto?". | El valor persistido pertenece al conjunto {si,no}. | Prueba funcional de enum. |

| RF-09 | El sistema debera permitir observaciones afectivas opcionales. | Se visualiza en UI y se guarda en BD. | Prueba de integracion FE-BE-BD. |

| RF-10 | El sistema debera dividir la encuesta en 3 pasos: generales, descriptiva y afectiva. | El flujo muestra exactamente 3 pasos. | Prueba E2E del flujo multi-step. |

| RF-11 | El sistema debera mantener los datos al navegar entre pasos sin perder estado. | Volver/avanzar conserva el 100% de campos cargados. | Prueba E2E de persistencia de estado. |

| RF-12 | El sistema debera permitir avanzar y retroceder con botones Continuar/Volver. | Ambos botones funcionan en pasos intermedios. | Prueba UI de navegacion por botones. |

| RF-13 | El sistema debera mostrar confirmacion explicita luego de envio exitoso. | El mensaje de exito aparece solo tras respuesta 2xx. | Prueba funcional con mock de respuesta. |

| RF-14 | El sistema debera almacenar las respuestas en base de datos relacional. | Cada envio exitoso genera un registro en `encuestas`. | Prueba de integracion con consulta SQL. |



| RF-15 | El sistema debera procesar respuestas para generar estadisticas agregadas. | El endpoint de estadisticas devuelve KPIs y distribuciones consistentes. | Prueba API de consistencia de calculos. |

| RF-16 | El sistema debera mostrar dashboard con total, completadas, puntaje global, satisfaccion y distribuciones por dieta y sexo. | Todos los indicadores se renderizan con datos reales. | Prueba UI + API de dashboard. |

| RF-17 | El sistema debera mostrar grafico radar de atributos descriptivos. | El radar presenta los 6 atributos con valores agregados. | Prueba visual + validacion de dataset. |

| RF-18 | El sistema debera permitir seleccionar/deseleccionar atributos del radar y recalcular visualizacion. | Al ocultar un atributo, el radar se redibuja sin ese eje. | Prueba UI interactiva del radar. |

| RF-19 | El sistema debera mostrar tooltips de metricas al pasar el cursor por graficos. | Tooltip visible con etiqueta y valor en graficos clave. | Prueba UI de hover por grafico. |

| RF-20 | El sistema debera exportar estadisticas en PDF y Excel. | Ambos archivos se descargan sin error y con contenido valido. | Prueba funcional de exportaciones. |

| RF-21 | El sistema debera permitir refrescar estadisticas mediante accion explicita. | El boton refresco reconsulta datos y actualiza UI. | Prueba UI/API con cambio de datos. |

| RF-22 | El sistema debera permitir alternar tema claro/oscuro. | El cambio de tema aplica en la interfaz completa. | Prueba UI de toggle de tema. |

| RF-23 | El sistema debera adaptarse a escritorio, tablet y movil sin perdida funcional. | No hay desbordes ni bloqueo de acciones criticas. | Prueba responsive en 3 resoluciones objetivo. |

| RF-24 | El sistema debera validar campos obligatorios antes de avanzar o enviar. | Se muestran errores por campo y se bloquea avance/envio. | Prueba funcional de validaciones. |

| RF-25 | El sistema debera proveer feedback visual de carga, error y exito en interacciones criticas. | Estados visuales presentes en envio y carga de dashboard. | Prueba UI de estados de interfaz. |

## 8.2 Requerimientos No Funcionales (21)

Nota de lectura: esta tabla resume los RNF en formato tecnico. El desarrollo academico completo (justificacion y riesgos por RNF) se detalla en la Seccion 7.

| ID | Requerimiento normalizado | Criterio de aceptacion | Prueba asociada |

|---|---|---|---|

| RNF-01 | El 95% de las operaciones de carga de encuesta, cambio de pagina y actualizacion de estadisticas debera responder en <=3s en condiciones normales. | p95 <=3s en medicion de 15 min. | Prueba de performance (k6 + Web Vitals). |

| RNF-02 | Las imagenes principales del Home deberan cargarse en <2s con formatos optimizados web. | LCP de Home <2s para assets principales. | Lighthouse/WebPageTest con perfil movil. |

| RNF-03 | El sistema debera aceptar al menos 30 envios concurrentes sin perdida de informacion ni caida de servicio. | Error rate <1% durante 10 min de carga. | Prueba de carga concurrente. |

| RNF-04 | El sistema debera permitir incorporar nuevas preguntas/metricas sin rediseñar la arquitectura base. | Alta de nuevo campo con cambios acotados (<=3 modulos). | Prueba de cambio controlado. |

| RNF-05 | El sistema debera estar operativo durante la jornada de evaluacion definida. | Disponibilidad >=99.5% en franja acordada. | Monitoreo por healthcheck. |

| RNF-06 | El sistema debera persistir correctamente respuestas para consulta posterior. | >=99.9% de 2xx persistidos integramente. | Conciliacion API vs BD. |

| RNF-07 | Ante fallo de envio, el sistema debera informar error y permitir reintento sin perdida de datos. | Reintento exitoso con datos intactos. | Prueba de resiliencia con falla de red simulada. |

| RNF-08 | El sistema debera almacenar solo respuestas validas y consistentes con escalas/opciones definidas. | 0 registros fuera de dominio en BD. | Pruebas de validacion DTO + constraints SQL. |

| RNF-09 | El panel administrativo debera requerir autenticacion obligatoria mediante Clerk. | 100% de endpoints admin rechazan anonimicos. | Pruebas de seguridad de autenticacion. |

| RNF-10 | Solo usuarios administradores autorizados podran acceder a estadisticas y funciones admin. | 100% de endpoints admin validan rol. | Pruebas de autorizacion por rol. |

| RNF-11 | La encuesta publica no debera requerir nombre, correo ni contrasena. | Flujo de encuesta completo sin login ni PII obligatoria. | Prueba funcional de anonimato. |

| RNF-12 | El sistema debera proteger datos de encuestas, usuarios e imagenes frente a accesos no autorizados. | 0 secretos expuestos y controles de acceso activos. | Auditoria de seguridad + revision de configuracion. |

| RNF-13 | La encuesta debera completarse de forma intuitiva mediante flujo multi-step claro. | Tasa de finalizacion >=85% en muestra objetivo. | Test de usabilidad moderado. |

| RNF-14 | La interfaz debera alcanzar Lighthouse Accessibility >=70/100 en vistas clave. | Score >=70 en Home, Encuesta y Admin. | Auditoria Lighthouse versionada. |

| RNF-15 | La aplicacion debera adaptarse a movil, tablet y desktop sin perdida funcional. | 100% de casos criticos pasan en 3 breakpoints. | Suite responsive manual/automatizada. |

| RNF-16 | La interfaz debera mantener paleta oficial y coherencia visual entre vistas. | 100% de vistas usan tokens de disenio oficiales. | Revision de estilos/tokens. |

| RNF-17 | El sistema debera funcionar en ultimas versiones de Chrome, Edge y Firefox. | 0 defectos bloqueantes P1 por navegador objetivo. | Matriz de compatibilidad cross-browser. |

| RNF-18 | Frontend, backend y BD deberan intercambiar informacion de forma consistente. | Error de contrato API <0.5% diario. | Pruebas de contrato + integracion E2E. |

| RNF-19 | El codigo debera organizarse en modulos/componentes/servicios reutilizables. | Cumplimiento de arquitectura modular definida. | Revision tecnica + metricas de calidad. |

| RNF-20 | El sistema debera desplegarse y operar en Vercel/Render/Supabase sin depender del entorno local. | Deploy reproducible + smoke tests OK. | Prueba CI/CD en entorno objetivo. |

| RNF-21 | El sistema debera registrar eventos relevantes (accesos admin, errores y fallos) para monitoreo. | 100% de eventos criticos logueados con trazabilidad. | Prueba de observabilidad y auditoria de logs. |

## 7. Anexo B - RNF Completos con Estructura Academica (ISO 25010 + IEEE 29148)

En esta seccion se documentan los 21 RNF del proyecto con la estructura solicitada por catedra:

- Nombre del requerimiento no funcional.
- Grupo o categoria.
- Descripcion formal del requerimiento.
- Justificacion de relevancia para el escenario.
- Dos riesgos asociados (cada uno con descripcion, impacto y mitigacion).

| ID | Nombre del RNF | Grupo o categoria | Descripcion formal del requerimiento | Justificacion | Riesgo 1 (descripcion / impacto / mitigacion) | Riesgo 2 (descripcion / impacto / mitigacion) |

|---|---|---|---|---|---|

| RNF-01 | Tiempo de respuesta del sistema | Rendimiento, capacidad y escalabilidad | El 95% de las operaciones de carga de encuesta, cambio de pagina y actualizacion de estadisticas debera responder en  $\leq 3$  segundos en condiciones normales. | Mantiene la experiencia fluida y evita abandono en campo. | Latencia alta / abandono o respuestas apuradas / optimizar consultas, cache y payloads. | Criterio de medicion ambiguo / conflictos en aceptacion / fijar p95, entorno y ventana de prueba. |

| RNF-02 | Carga eficiente de imagenes | Rendimiento, capacidad y escalabilidad | Las imagenes principales del Home deberan cargar en  $< 2$  segundos usando formatos optimizados (WebP/JPG comprimido). | El Home es puerta de entrada y afecta percepcion de calidad. | Imagenes pesadas / demora inicial / pipeline de compresion automatica. | Alto consumo de datos moviles / menor participacion / versiones responsive por dispositivo. |

| RNF-03 | Capacidad para multiples respuestas | Rendimiento, capacidad y escalabilidad | El sistema debera soportar al menos 30 envios concurrentes de encuestas sin perdida de informacion ni interrupcion del servicio. | Durante jornadas puede haber multiples jurados simultaneos. | Saturacion del backend / perdidas de encuestas / tuning de pool DB y pruebas de carga. | Timeouts en picos / datos incompletos / reintentos idempotentes y colas de procesamiento. |

| RNF-04 | Escalabilidad funcional | Rendimiento, capacidad y escalabilidad | El sistema debera permitir agregar nuevas preguntas, metricas o graficos sin rediseñar completamente la arquitectura existente. | El proyecto academico puede evolucionar en nuevas iteraciones. | Alto acoplamiento / cambios lentos y caros / modularizacion por dominios. | Regresiones al extender funcionalidad / inestabilidad / pruebas de contrato y regresion. |

| RNF-05 | Disponibilidad durante la evaluacion | Disponibilidad, confiabilidad y recuperacion | El sistema debera mantener disponibilidad  $\geq 99.5\%$  durante la franja operativa acordada de jornada sensorial. | Una caida en horario de uso invalida la toma de datos. | Caida total del servicio / interrupcion de actividad / monitoreo y alertas proactivas. | Dependencia de un solo punto / indisponibilidad extendida / plan de contingencia y failover. |

| RNF-06 | Persistencia de respuestas | Disponibilidad, confiabilidad y recuperacion | Al menos 99.9% de encuestas con respuesta HTTP 2xx deberan quedar persistidas integramente en base de datos. | La confiabilidad de resultados depende de persistencia real. | Exitos visual sin guardado real / datos faltantes / confirmar commit antes del mensaje de exito. | Escritura parcial de campos / estadisticas sesgadas / transacciones y constraints. |

| RNF-07 | Recuperacion ante fallos de envio | Disponibilidad, confiabilidad y recuperacion | Ante un error de envio, el sistema debera informar la falla y permitir reintento sin perdida de datos ingresados. | Evita frustracion y reduce perdida de informacion por red inestable. |

Perdida de estado del formulario / abandono / persistencia temporal local. | Duplicado por multiples intentos / distorsion de resultados / idempotencia por requestId. |

| RNF-08 | Integridad de datos | Disponibilidad, confiabilidad y recuperacion | El sistema debera almacenar unicamente respuestas validas segun opciones y escalas definidas en la encuesta. | Sin integridad no hay analisis confiable. | Datos fuera de rango / analisis incorrecto / validacion DTO + constraints SQL. | Inconsistencia entre frontend y backend / errores silenciosos / contratos de datos versionados. |

| RNF-09 | Autenticacion administrativa | Seguridad, privacidad y proteccion de datos | El acceso al panel administrativo debera requerir autenticacion obligatoria mediante Clerk y token valido. | Protege informacion interna del proyecto. | Acceso anonimo por configuracion defectuosa / exposicion de datos / guards obligatorios en rutas admin. | Sesiones comprometidas / uso indebido / expiracion y validacion estricta de tokens. |

| RNF-10 | Autorizacion por rol | Seguridad, privacidad y proteccion de datos | Solo usuarios con rol `admin` y estado `activo=true` podran acceder a estadisticas y funciones administrativas. | Separa responsabilidades entre encuestado y administrador. | Usuario sin permisos accede / fuga de informacion / chequeo de rol en backend y DB. | Control parcial por endpoint / bypass funcional / matriz de permisos y pruebas negativas. |

| RNF-11 | Encuesta anonima | Seguridad, privacidad y proteccion de datos | La encuesta publica no debera requerir nombre, correo ni contrasena para su completitud. | Reduce friccion de participacion y protege privacidad del encuestado. | Solicitud accidental de PII / riesgo legal-etico / formulario sin campos personales obligatorios. | Vinculacion indebida de respuestas / sesgo en resultados / politica estricta de anonimato. |

| RNF-12 | Proteccion de datos almacenados | Seguridad, privacidad y proteccion de datos | Datos de encuestas, usuarios e imagenes deberan protegerse contra accesos no autorizados y modificaciones indebidas. | Es clave para confianza y continuidad academica. | Filtracion de secretos / compromiso de entorno / manejo por variables de entorno y rotacion. | Acceso indebido a recursos almacenados / alteracion de datos / politicas de acceso minimo y auditoria. |

| RNF-13 | Usabilidad de la encuesta | Usabilidad, accesibilidad y experiencia de usuario | La encuesta debera poder completarse de forma intuitiva mediante flujo multi-step con navegacion clara y consistente. | Mejor usabilidad mejora tasa de finalizacion. | Flujo confuso / abandono alto / simplificar etiquetas y orden de campos. | Feedback insuficiente / errores reiterados / validaciones contextuales y mensajes claros. |

| RNF-14 | Accesibilidad visual | Usabilidad, accesibilidad y experiencia de usuario | La interfaz debera alcanzar  $\geq 70/100$  en Lighthouse Accessibility en Home, Encuesta y Dashboard. | Garantiza inclusion minima y calidad visible. | Bajo contraste / exclusion de usuarios / ajuste de paleta y chequeo WCAG. | Falta de evidencias formales / rechazo en evaluacion / auditorias versionadas por release. |

| RNF-15 | Diseño responsive | Usabilidad, accesibilidad y experiencia de usuario | La aplicación deberá adaptarse a móvil, tablet y desktop sin desbordes ni pérdida funcional en casos críticos. | La recolección ocurre en dispositivos heterogéneos. | Ruptura de layout en móvil / imposibilidad de encuestar / pruebas por breakpoints objetivo. | Dashboard ilegible en pantallas chicas / decisiones erradas / reorganización adaptativa de gráficos. |

| RNF-16 | Consistencia visual | Usabilidad, accesibilidad y experiencia de usuario | La interfaz deberá mantener paleta oficial NutriLen y coherencia visual entre Home, Encuesta y Administrador. | Refuerza identidad y reduce carga cognitiva. | Inconsistencias de UI / percepción no profesional / uso obligatorio de tokens de diseño. | Colores hardcodeados dispersos / mantenimiento difícil / centralizar tema en variables globales. |

| RNF-17 | Compatibilidad con navegadores | Compatibilidad, interoperabilidad e integración | El sistema deberá funcionar en últimas versiones estables de Chrome, Edge y Firefox sin defectos bloqueantes. | Asegura acceso amplio sin dependencia de un navegador único. | Fallas en navegador específico / exclusión de usuarios / matriz de pruebas cross-browser. | APIs no soportadas / errores en cliente / polyfills y lint de compatibilidad. |

| RNF-18 | Integración frontend-backend-BD | Compatibilidad, interoperabilidad e integración | Frontend, backend y base de datos deberán intercambiar información de forma consistente según contratos de API y DTO definidos. | Evita inconsistencias entre captura y analítica. | Campos desalineados entre capas / errores de guardado / contratos versionados y validación estricta. | Cambios no coordinados / roturas en producción / pruebas E2E y de contrato por CI. |

| RNF-19 | Mantenibilidad del código | Mantenibilidad, modularidad y deuda técnica | El código deberá organizarse en módulos, componentes y servicios reutilizables para facilitar mantenimiento y evolución. | Reduce deuda técnica y costo de cambios futuros. | Código acoplado / alta complejidad / límites de complejidad y refactor continuo. | Duplicación excesiva / errores repetidos / extracción de utilidades compartidas. |

| RNF-20 | Despliegue y operación | Portabilidad, despliegue y operación | El sistema deberá poder desplegarse y operar en Vercel/Render/Supabase sin depender del entorno local del desarrollador. | Garantiza reproducibilidad y continuidad operativa. | Funciona solo local / bloqueo de entrega / pipelines CI/CD con smoke tests. | Configuración divergente por entorno / fallas intermitentes / checklist de variables y secretos. |

| RNF-21 | Registro de eventos del sistema | Observabilidad, auditoría, cumplimiento y gobierno | El sistema deberá registrar accesos admin, errores de envío y fallos operativos con trazabilidad (timestamp, endpoint, requestId). | Permite monitoreo, auditoría y diagnóstico de incidentes. | Incidente sin evidencia / MTTR elevado / logging estructurado centralizado. | Retención insuficiente de eventos / auditoría incompleta / política de retención y respaldo de logs. |

### 7.1 Verificación de cobertura

- Total de requerimientos no funcionales documentados: 21.
- Riesgos por RNF: 2.
- Total de riesgos analizados: 42.
- Cobertura de categorías: representadas las 8 categorías solicitadas.
- Integración aplicada: redacción formal IEEE 29148 + clasificación ISO/IEC 25010 + gestión de riesgos por requerimiento.